

Spring Core and Maven

1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.library</groupId>
```

```
<artifactId>LibraryManagement</artifactId>
```

```
<version>1.0</version>
```

```
<properties>
```

```
    <maven.compiler.source>1.8</maven.compiler.source>
```

```
    <maven.compiler.target>1.8</maven.compiler.target>
```

```
    <spring.version>5.3.30</spring.version>
```

```
</properties>
```

```
<dependencies>
```

```
    <dependency>
```

```
        <groupId>org.springframework</groupId>
```

```
        <artifactId>spring-context</artifactId>
```

```
        <version>${spring.version}</version>
```

```
    </dependency>
```

```
<dependency>
```

```
    <groupId>org.springframework</groupId>
```

```
    <artifactId>spring-aop</artifactId>  
    <version>${spring.version}</version>  
</dependency>
```

```
<dependency>  
    <groupId>org.springframework</groupId>  
    <artifactId>spring-webmvc</artifactId>  
    <version>${spring.version}</version>  
</dependency>
```

```
<dependency>  
    <groupId>org.aspectj</groupId>  
    <artifactId>aspectjweaver</artifactId>  
    <version>1.9.22</version>  
</dependency>
```

```
</dependencies>
```

```
<build>  
    <plugins>  
        <plugin>  
            <groupId>org.apache.maven.plugins</groupId>  
            <artifactId>maven-compiler-plugin</artifactId>  
            <version>3.11.0</version>  
            <configuration>  
                <source>1.8</source>  
                <target>1.8</target>  
            </configuration>
```

```
        </plugin>
    </plugins>
</build>
</project>
```

2. BookRepository.java

```
package com.library.repository;

import org.springframework.stereotype.Repository;

@Repository
public class BookRepository {

    public void display() {

        System.out.println("Book Repository Method Executed");

    }

}
```

3. BookService.java

This demonstrates both **constructor injection** and **setter injection**.

```
package com.library.service;

import org.springframework.stereotype.Service;
import com.library.repository.BookRepository;

@Service
public class BookService {

    private BookRepository bookRepository;

    public BookService() {

    }

    public BookService(BookRepository bookRepository) {

        this.bookRepository = bookRepository;

        System.out.println("Constructor Injection Executed");

    }

}
```

```
public void setBookRepository(BookRepository bookRepository) {  
    this.bookRepository = bookRepository;  
    System.out.println("Setter Injection Executed");  
}
```

```
public void issueBook() {  
    System.out.println("Book Service Method Executed");  
    bookRepository.display();  
}  
}
```

4. LoggingAspect.java

```
package com.library.aspect;  
  
import org.aspectj.lang.ProceedingJoinPoint;  
import org.aspectj.lang.annotation.*;  
import org.springframework.stereotype.Component;  
  
@Aspect  
@Component  
public class LoggingAspect {  
  
    @Before("execution(* com.library.service.*.*(..))")  
    public void beforeMethod() {  
        System.out.println("Before Method Execution");  
    }  
  
    @After("execution(* com.library.service.*.*(..))")  
    public void afterMethod() {  
        System.out.println("After Method Execution");  
    }  
  
    @Around("execution(* com.library.service.*.*(..))")  
    public Object logExecutionTime(ProceedingJoinPoint joinPoint)  
        throws Throwable {
```

```

    long start = System.currentTimeMillis();

    Object result = joinPoint.proceed();

    long end = System.currentTimeMillis();

    System.out.println(
        joinPoint.getSignature().getName()
            + " executed in "
            + (end - start)
            + " ms");

    return result;
}
}

```

5. applicationContext.xml

Create inside:

src/main/resources/applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:context="http://www.springframework.org/schema/context"
    xmlns:aop="http://www.springframework.org/schema/aop"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd

        http://www.springframework.org/schema/context
        https://www.springframework.org/schema/context/spring-context.xsd

        http://www.springframework.org/schema/aop
        https://www.springframework.org/schema/aop/spring-aop.xsd">

    <!-- Component Scan -->
    <context:component-scan base-package="com.library"/>

```

```

<!-- Enable AOP -->
<aop:aspectj-autoproxy/>

<!-- Repository Bean -->
<bean id="bookRepository"
      class="com.library.repository.BookRepository"/>

<!-- Service Bean -->
<bean id="bookService"
      class="com.library.service.BookService">

    <constructor-arg ref="bookRepository"/>
    <property name="bookRepository"
              ref="bookRepository"/>

</bean>

</beans>

```

6. Main Class

```

package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

import com.library.service.BookService;

public class LibraryManagementApplication {

    public static void main(String[] args) {

        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml");

        BookService service =
            context.getBean("bookService",
                BookService.class);

        service.issueBook();
    }
}

```

```
}  
}
```

Expected Output

Constructor Injection Executed

Setter Injection Executed

Before Method Execution

Book Service Method Executed

Book Repository Method Executed

issueBook executed in 0 ms

After Method Execution

Exercise 6 (Annotation Configuration)

Already completed using:

@Service

@Repository

@Aspect

@Component

<context:component-scan>

Exercise 9 — Spring Boot Application

Create using **Spring Initializr** with dependencies:

- Spring Web
- Spring Data JPA
- H2 Database

application.properties

spring.datasource.url=jdbc:h2:mem:testdb

spring.datasource.driverClassName=org.h2.Driver

spring.jpa.hibernate.ddl-auto=update

spring.h2.console.enabled=true

Book.java

```
package com.library.entity;
```

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
```

```
@Entity
public class Book {

    @Id
    private int id;
    private String name;

    public Book() {
    }

    public Book(int id, String name) {
        this.id=id;
        this.name=name;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id=id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name=name;
    }
}
```

BookRepository.java

```
package com.library.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.library.entity.Book;
```



```
public interface BookRepository
    extends JpaRepository<Book,Integer> {
}
```

BookController.java

```
package com.library.controller;
```

```
import java.util.List;
```

```
import org.springframework.beans.factory.annotation.Autowired;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import com.library.entity.Book;
```

```
import com.library.repository.BookRepository;
```

```
@RestController
```

```
@RequestMapping("/books")
```

```
public class BookController {
```

```
    @Autowired
```

```
    private BookRepository repository;
```

```
    @PostMapping
```

```
    public Book addBook(@RequestBody Book book) {
```

```
        return repository.save(book);
```

```
    }
```

```
    @GetMapping
```

```
    public List<Book> getBooks() {
```

```
        return repository.findAll();
```

```
    }
```

```
}
```

Main Class

```
package com.library;
```

```
import org.springframework.boot.SpringApplication;
```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
```

```
public class LibraryManagementApplication {
```

```
public static void main(String[] args) {  
    SpringApplication.run(  
        LibraryManagementApplication.class,  
        args);  
}  
}
```

Testing

Add Book

POST <http://localhost:8080/books>

Body:

```
{  
  "id":1,  
  "name":"Spring Framework"  
}
```

Get Books

GET <http://localhost:8080/books>

- ▼  LibraryManagement
 - ▼  src/main/java
 - ▼  com.library
 - >  LibraryManagementApplication.java
 - ▼  com.library.aspect
 - >  LoggingAspect.java
 - ▼  com.library.repository
 - >  BookRepository.java
 - ▼  com.library.service
 - >  BookService.java
 - >  src/main/resources
 - >  JRE System Library [JavaSE-1.8]
 - >  Maven Dependencies
 -  target/generated-sources/annotations
 - >  bin
 - >  src
 - >  target
 -  pom.xml

-
- The screenshot shows the Eclipse IDE with the following components:
- Package Explorer:** Shows the project structure with packages like 'com.library', 'com.library.aspect', 'com.library.repository', and 'com.library.service'.
 - Main Editor:** Displays the code for 'LibraryManagementApplication.java'. The code is as follows:


```

package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Service;

public class LibraryManagementApplication {

    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext(
                "applicationContext.xml");

        BookService service =
            context.getBean("bookService",
                BookService.class);

        service.issueBook();
    }
}

```
 - Task List:** Shows a list of tasks with checkboxes for 'Saturday - Today', 'This Week', 'Next Sunday', 'Next Monday', 'Next Tuesday', 'Next Wednesday', 'Next Thursday', 'Next Friday', 'Next Saturday', 'Next Week', 'Two Weeks', 'Future', 'Outgoing', 'Incoming', 'Unscheduled', and 'Completed'.
 - Outline:** Shows the 'LibraryManagementApplication' class and its 'main(String[]) void' method.
 - Problems:** Shows a warning about a 'terminated' library management application.
 - Console:** Shows the output of the application, including the message 'IssueBook executed in 27 ms'.







